gateway-api: the customer edge and the React SPA

announce-reactive, worktree feat/gateway-api, 2026-09-26. What was built, how to build, run and use it, and what was verified.

1. What this module is

gateway-api is the customer-facing edge of the announce marketplace: one Spring Boot jar that is both the API gateway (Spring Cloud Gateway on WebFlux, port 8080) and the host of the single-page application **@announce/customer-web** (React 18, TypeScript). It replaces the old **gateway** module (port 9001), which is deleted. It follows the design in **doc/frontend-edges.md** and the litemall model (litemall-gateway-api).

- **Identity at the edge.** Members log in against **announce_user**; the edge issues its own RS256 access token (2 h) and a rotating refresh token (30 days) kept only as a SHA-256 hash in the new table **announce_refresh_token** and delivered as an httpOnly cookie on **/auth**.
- **Deny by default.** Every request needs a valid Bearer token unless **PublicPaths** lists it (catalog reads, search, suggest, click-out, provider webhooks, the SPA shell).
- **Trusted headers.** Client-supplied X-User-* and X-Seller-Id are stripped; the edge injects X-User-Id, X-User-Type=MEMBER and (on announce-management routes) X-Seller-Id, plus X-Service-Token (an HMAC) so backends can later refuse traffic that did not pass an edge.
- **Explicit routes, no StripPrefix.** announce-management, order, messaging (REST and WebSocket) and the OCS searcher/suggest (SetPath to the tenant API of **announce_index**).
- **SPA history fallback.** Deep links such as /announce/abc serve index.html; missing assets stay 404.

2. Layout of the module

Path	Content
gateway-api/pom.xml	Parent org.joviste:announce. Gateway, Eureka, config client, reactive security, spring-security-oauth2-jose (Nimbus), db-reactive, R2DBC MySQL, MockWebServer for tests. Profile **webapp** (active by default): frontend-maven-plugin installs node v20.16.0 / npm 10.8.1 under target/, runs npm ci and npm run build.
src/main/java/.../gatewayapi/auth	AuthController, AuthDtos, ports UserAccounts and RefreshTokenStore, R2dbcUserAccounts, R2dbcRefreshTokenStore, RefreshTokenService, AccessTokens, RefreshCookies, AuthException(+Handler), AuthProperties, ClockConfig.
.../security	SecurityConfig, PublicPaths, IdentityForwardingFilter, WebSocketTokenFilter.
.../machine	MachineTokenProperties, MachineTokens (mint/verify), MachineTokenRelayFilter.
.../gateway	SellerHeaderGatewayFilterFactory (route filter "SellerHeader").
.../web	SpaHistoryFallbackFilter, ProblemDetails.
.../config	JwtProperties, RsaKeys (PEM or ephemeral), JwtConfig (encoder, decoder, BCrypt).
src/main/resources/config/application.yml	Port, routes, R2DBC, Eureka, optional config server, announce.* properties, the DEVELOPMENT keypair.
src/main/webapp	The SPA: package.json, webpack/, tsconfig, jest.conf.js, i18next-parser.config.js, public/, app/.
src/test/java	EdgeRoutesTest (MockWebServer), MachineTokensTest, AuthPersistenceTest (live MySQL), InMemoryAuthStores.
db-reactive	announce_refresh_token DDL in sql/announce_table.sql (+ regenerated announce_reactive_db.sql), generated reactive mapper AnnounceRefreshTokenMapper, AnnounceRefreshTokenMapperTest.

3. How to build

```
# once per machine: parent pom and db-reactive (new mapper) in the local repository
mvn -N install
mvn -pl db-reactive install -DskipTests

# edge only (no node needed), with the tests
mvn -pl gateway-api -P '!webapp' test

# edge + SPA: the jar that serves the SPA at /
mvn -pl gateway-api -P webapp -DskipTests package
#   -> gateway-api/target/gateway-api-0.0.1-SNAPSHOT.jar

# SPA alone
cd gateway-api/src/main/webapp
npm ci && npm test && npm run i18n:check && npm run build
```

Maven runs on /usr/lib/jvm/java-21-openjdk-amd64 (java on PATH is 11). The frontend-maven-plugin installs its own node under gateway-api/target/, so the machine needs no node for the Maven build; the local nvm node v20.16.0 works for the npm commands.

4. How to run

With Eureka (8761) and the config server (8888) running, the jar needs no arguments: routes resolve lb://announce-management, lb://order and lb://messaging through discovery.

```
java -jar gateway-api/target/gateway-api-0.0.1-SNAPSHOT.jar
```

Without Eureka, or when 8080 is taken (on the development machine a Jenkins holds 8080), point the routes at URLs and pick another port:

```
java -jar gateway-api/target/gateway-api-0.0.1-SNAPSHOT.jar --server.port=8090 \
  --eureka.client.enabled=false --spring.cloud.config.enabled=false \
  --announce.routes.announce-management=http://localhost:8085 \
  --announce.routes.order=http://localhost:8087 --announce.routes.messaging=http://localhost:8086 \
  --announce.routes.messaging-ws=ws://localhost:8086
```

Prerequisites for a full local run: the native MySQL with announce_reactive_db (run sql/announce_reactive_db.sql once, it now includes announce_refresh_token), docker/cqrs up (EventSourcingDB, Kafka, Elasticsearch, OCS indexer 8535, searcher 8534, suggest 8536), announce-management on 8085 (mvn -pl announce-management -P '!webapp' spring-boot:run). The SPA dev server (npm run webapp:dev, port 9000) proxies /api, /auth, /out and /ws to EDGE_URL (default http://localhost:8080).

Properties to set in production (environment variables override them):

Property	Meaning	Default
announce.jwt.private-key-pem / public-key-pem	RS256 keypair (PKCS#8 / X.509 PEM). Blank = ephemeral pair (tokens die with the process).	DEVELOPMENT keypair in application.yml
announce.jwt.issuer / audience / access-ttl	Claims of the access token	announce-customer / announce-customer-api / 2h
announce.auth.cookie-name / cookie-secure / refresh-ttl	Refresh cookie (httpOnly, path /auth, SameSite=Strict)	announce_refresh / false / 30d
announce.machine.service / secret	X-Service-Token HMAC	gateway-api / announce-dev-machine-secret
announce.search.searcher-url / suggest-url / index	OCS targets and tenant	http://localhost:8534 / :8536 / announce_index
announce.routes.announce-management|order|messaging|messaging-ws	Route targets	lb://... (Eureka)

Property	Meaning	Default
spring.r2dbc.mybatis.*	announce_reactive_db connection (ANNOUNCE_R2DBC_URL, ANNOUNCE_DB_USER, ANNOUNCE_DB_PASSWORD)	localhost:3306, announce

5. Routes and who may call them

Path	Target	Auth	Notes
/auth/login, /register, /refresh, /logout	edge (AuthController)	public	/auth/me needs a member
GET /api/announces/**, /api/categories/**, /api/brands/**	announce-management	public	SellerHeader copies X-User-Id into X-Seller-Id when present
POST/PUT /api/announces/**	announce-management	member	writes; X-Seller-Id = the member
GET /out/**	announce-management	public	302 to the crawled listing, 410 when GONE, 404 unknown
GET /api/search	searcher-url	public	SetPath /search-api/v1/search/announce_index; query string passes through
GET /api/suggest	suggest-url	public	SetPath /suggest-api/v1/announce_index/suggest
/api/auctions/**, /api/orders/**, /api/sellers/**	order	member	
POST /api/payments/webhook/**, /api/shipping/webhook/**	order	public	provider-signed; the service verifies
/api/conversations/**	messaging	member	
/ws/conversations/**	messaging (ws)	member	token as ?access_token= at handshake, injected as headers
everything else (GET/HEAD, no dot in the last segment)	index.html	public	SPA history fallback

Search request syntax (forwarded as is to OCS): q, limit, offset, sort=-field, term filters field=a,b, ranges field=min,max, withFacets=true. Response: slices[0].{matchCount, hits[].document.{id,data}, facets[]} and sortOptions[]. The SPA keeps its own URL (field=a,b for terms, field=min..max for ranges, page) and translates.

6. The auth contract (for other clients)

Call	Request	Response
POST /auth/register	{username (3-63, [A-Za-z0-9._@-]), password (8-63), nickname?, mobile?}	201 {accessToken, tokenType, expiresIn, user{id, username, nickname, mobile, avatar}} + Set-Cookie announce_refresh; 409 when taken
POST /auth/login	{username (or mobile), password}	200 same body + cookie; 401 bad credentials; 403 disabled
POST /auth/refresh	cookie only	200 new access token + rotated cookie; 401 unknown/revoked/expired (the old cookie is dead after a rotation)
POST /auth/logout	cookie	204, cookie cleared, token revoked
GET /auth/me	Bearer	200 user; 401 without
any failure		application/problem+json {status, title, detail, errors?}

7. The SPA

Phase 1 (works against the backend of today): shell with kind links, suggest box and language switch (en, da, sv); login / register / profile; home with the category roots per kind and the latest announces per kind; search with facets from the OCS response (term facets as checkboxes, intervals as min/max), sort, pagination, cards with a source badge that links crawled listings out through /out/{id}; announce detail with one layout per kind and a seller box; post wizard (kind, category of that kind, kind form with the aggregate's rules, location and picture

URLs, preview, save draft, publish; also edits); my announces with edit, publish, withdraw and mark sold.

Phase 2 (behind flags, default off): auction panel, buy box, orders list and detail (doc/order-context.md §8), contact seller, inbox and thread over the WebSocket with REST polling fallback (§9). Turn them on in the browser console; "mocks" starts MSW so the screens run without the order and messaging services:

```
localStorage.announceFeatures = '{"trade":true,"chat":true,"mocks":true}'   // then reload
```

Session model: the access token lives in memory (never in localStorage); on a 401 the axios instance refreshes once through the cookie (single flight, parallel 401s share it) and retries; when the refresh fails the session ends and guarded routes redirect to /login. A reload starts with a silent refresh (bootstrapSession).

Code map (src/main/webapp/app): config/ (store, http with the refresh interceptor, tokenStore, features), i18n/ (bundled JSON per namespace and language; npm run i18n:check fails on a missing key), shared/api (typed clients written from the controllers' DTOs), shared/model (DTO types and enums), shared/auth (slice, RequireAuth), shared/components, modules/{home, search, announce, post, my, auth, trade, chat}, mocks/.

8. What was verified

Suite	Result
mvn -pl db-reactive test (AllReactiveMappersSmokeTest + AnnounceRefreshTokenMapperTest)	41 tests green (39 mappers incl. the new one, 2 refresh-token tests)
mvn -pl gateway-api -P '!webapp' test	16 tests green: 13 route tests against MockWebServer, MachineTokensTest, 2 persistence tests on the live MySQL
mvn -pl gateway-api -P webapp -DskipTests package	jar built (node installed under target/, npm ci, webpack production build into target/classes/static)
npm test / npm run i18n:check	29 jest tests green; en, da, sv complete
Manual checklist (doc/frontend-edges.md §7, recorded in §9)	register, log in, post and publish one announce of each kind, event with end before start and job with AUCTION refused (409), write without token 401, each kind found in /api/search with its kind facet (source=local), crawled listings from dba_cars.json listed with source=dba, /out/{id} answers 302 to dba.dk and counts the click, withdraw, refresh rotation, logout revokes

9. Environment fixes made on the way (docker/cqrs)

- The suggest container is now published on host port **8536** (was 8081, the port reserved for gateway-admin).
- application.search-service.yml: **weightedFields** moved from under settings to the query level; the current ocs-search-service:latest image refused to start otherwise.
- The EventSourcingDB volume needed chown 100:101 (the image runs as user esdb); the container now starts healthy.
- A fresh Elasticsearch has no announce_index and the indexer's update API refuses to create one ("langcode required for new index"). Bootstrap once: GET :8535/indexer-api/v1/full/start/announce_index?locale=en, then POST :8535/indexer-api/v1/full/done with the returned session; after that announce-management's projections and the listings reindex work.
- Kafka: another compose project (litemall) already holds 9092; the announce services can use it, the announce_kafka container simply does not start while it is there.

10. Follow-ups

- announce-management should read X-User-Id; then drop the SellerHeader route filter.
- Backends verify X-Service-Token (MachineTokens.verify shows the rule: same secret, timestamp within a skew).

- Profile editing, image upload, SEO head and sitemap, a WebSocket mock for the chat screens.
- gateway-admin (worktree of the same name) duplicates the security package as litemall does.
- Rotate the committed development keypair before any deployment and set ANNOUNCE_MACHINE_SECRET.